Orange Digital Center

Digital IC Design

Group 2

Name: Youssef Ihab Mohamed

Final Project

Systolic Array Multiplication

➢ Systolic Array Matrix-Matrix Multiplication:

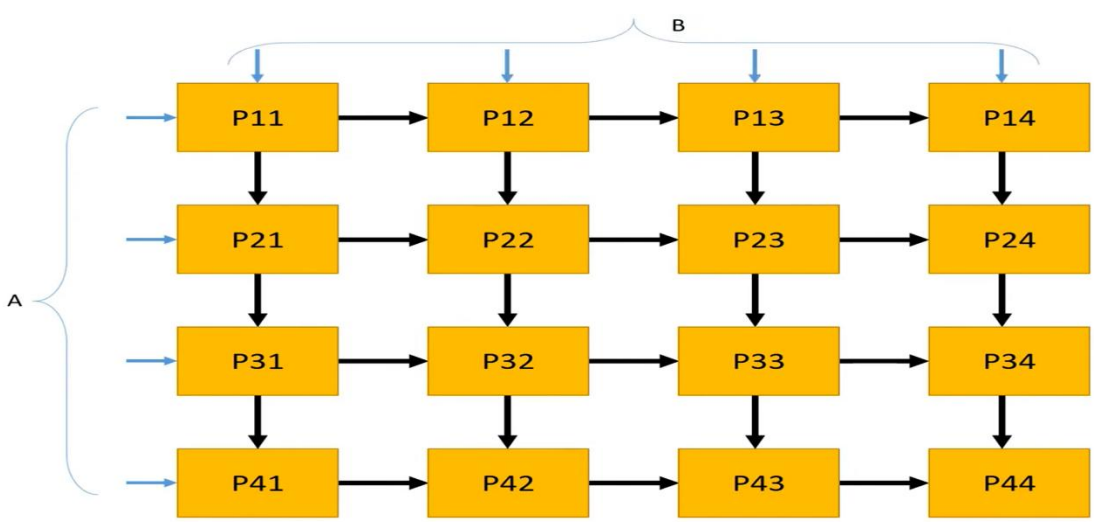
It differs from normal matrix-matrix multiplication significantly as systolic array uses parallel computation which reduces the number of operations needed to compute the multiplication, therefore it reduces the hardware utilization.

- To implement a Verilog code describing the architecture required for implementing the systolic array matrix-matrix multiplication, we need first to understand how the systolic array multiplication occurs.
- According to that, we will break the functionality of the systolic array into separate modules and discuss each one separately combining into the top module.

1st) Understanding the functionality of the systolic array:

Consider two 4x4 matrices:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \qquad B = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix}$$



➔ Each single module of those is called processing element and it's responsible for calculating multiplications and passing elements between processing elements and each other and that is our first module.

➔ Data is passed to the systolic array as follows:

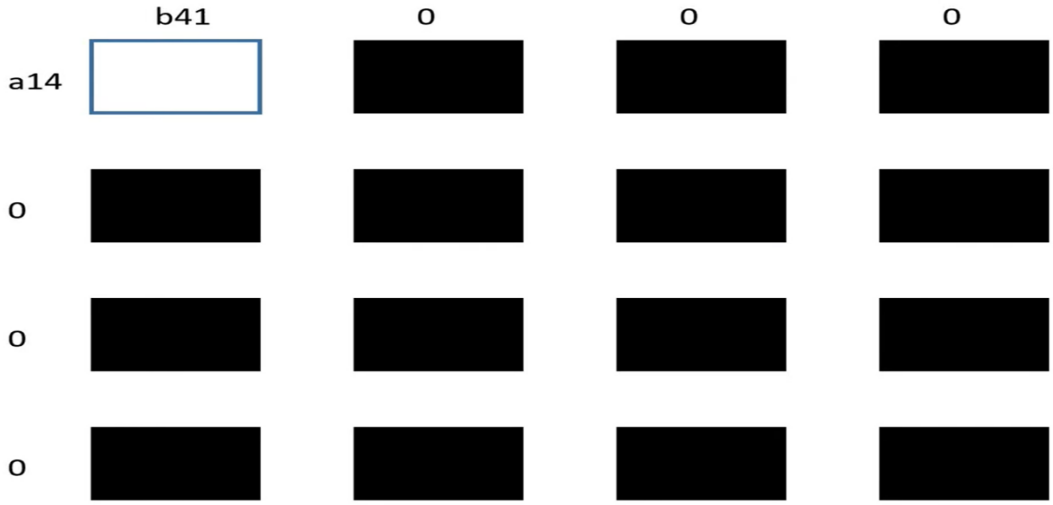
When Pi,j receives a and b, it performs the following steps:
1- Calculates a*b
2- Adds the result to the previous Ci,j and store the result in Ci,j
3- Sends a to Pi,j+1 unless j=4
4- Sends b to Pi+1,j unless i=4

			b14
		b13	b24
	b12	b23	b34
b11	b22	b33	b44
b31	b32	b43	0
b41	b42	0	0
b41	0	0	0

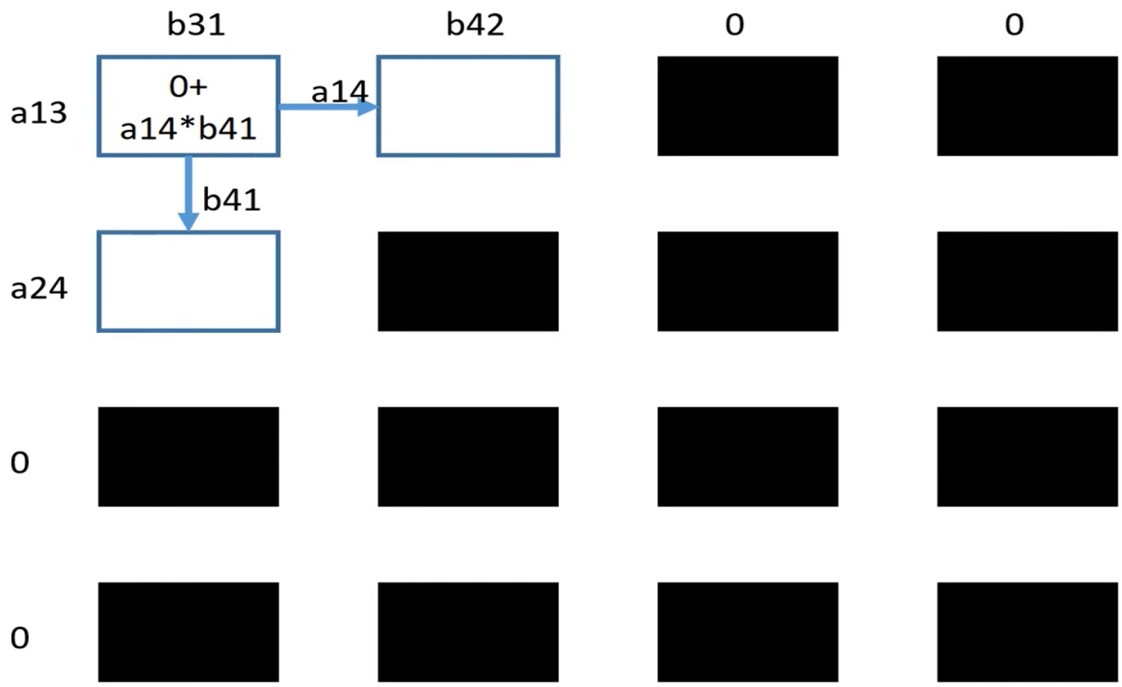
			a11	a12	a13	a14
		a21	a22	a23	a24	0
	a31	a32	a33	a34	0	0
a41	a42	a43	a44	0	0	0

Systolic Array

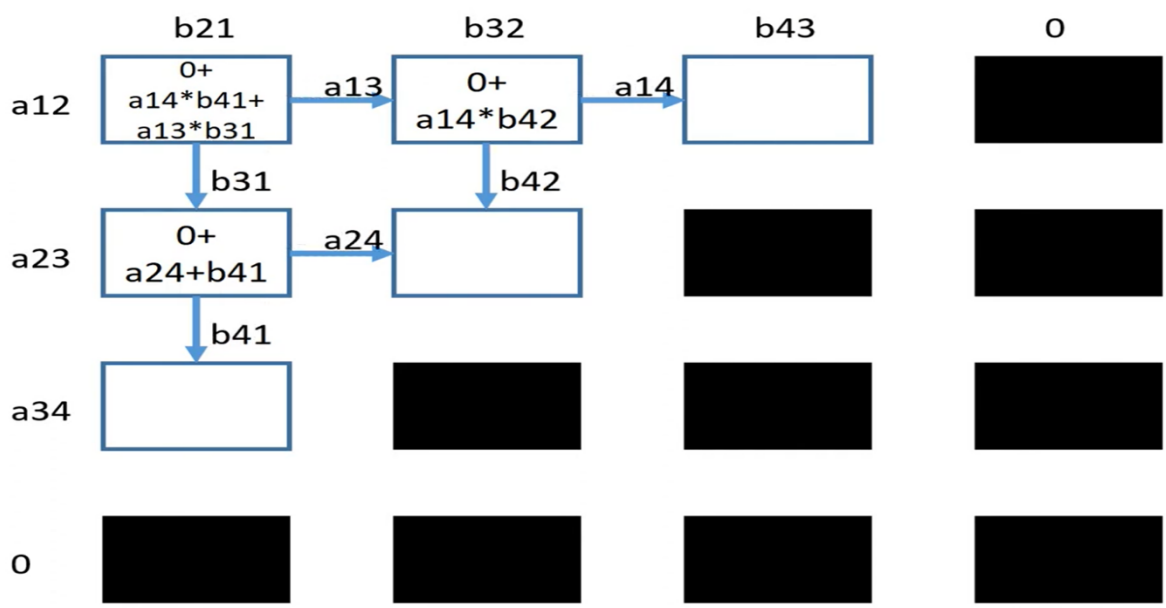
➔ Where each element of the systolic array is computed using each row and column of input data.



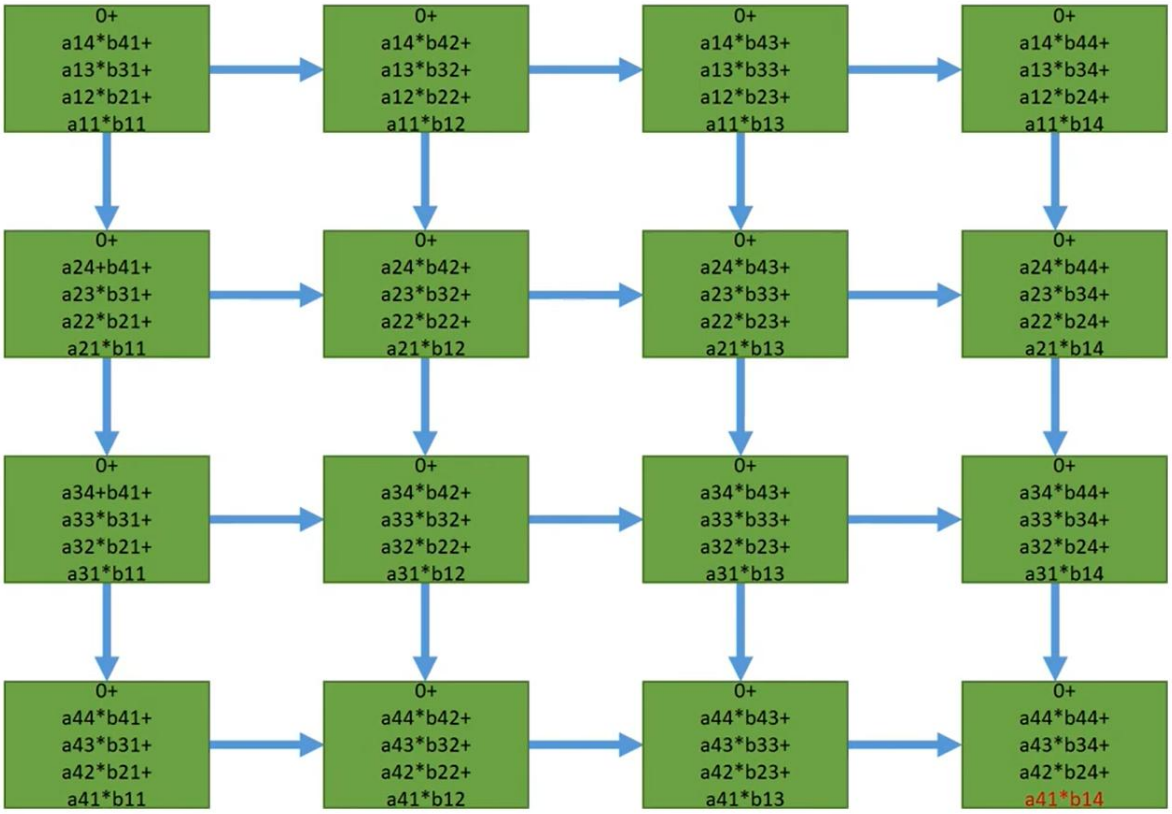
➔ Each element used for multiplication is then passed to the neighboring processing element.



➔ And that describes exactly the function of our 1st module which is handling the row-wise data flow, ensuring that matrix A values move horizontally, and matrix B values move vertically.



➔ The process continues till the last row and column data are processed and accumulated.



∴ **I will use Winograd algorithm for matrix multiplication to implement the systolic array multiplication breaking the top module into 3 submodules as follows:**

1) Row and column summations
2) Intermediate computations between processing elements
3) Final computations for processing elements and handling odd and even cases.

➔ Top module instantiating all submodules and calculating the final matrix.

➔ What is Winograd Algorithm for matrix multiplication?

1) The Winograd algorithm is an optimized approach to matrix multiplication that reduces the number of multiplications required, leveraging the idea of reusing intermediate computations. It is particularly useful for small matrix sizes and forms the basis for efficient convolution operations in deep learning.

2) Systolic arrays are hardware architectures that perform matrix operations efficiently by pipelining data through a network of interconnected processing elements (PEs). Winograd multiplication can be adapted for systolic arrays to further improve efficiency.

➔ Another Approach is to use processing element module for each individual processing element to multiply and accumulate but it's less efficient and have higher delay.

```
module processing_element #(
    parameter DATA_WIDTH = 16
)(
    input clk,
    input rst,
    input [DATA_WIDTH-1:0] A_in,
    input [DATA_WIDTH-1:0] B_in,
    input [2*DATA_WIDTH-1:0] sum_in,                // Accumulated sum from previous PE
    output reg [DATA_WIDTH-1:0] A_out,
    output reg [DATA_WIDTH-1:0] B_out,
    output reg [2*DATA_WIDTH-1:0] sum_out
);

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            sum_out <= 0;
            A_out <= 0;
            B_out <= 0;
        end else begin
            sum_out <= sum_in + (A_in * B_in); // Multiply-Accumulate
            A_out <= A_in;                     // Pass A to next PE in the row
            B_out <= B_in;                     // Pass B to next PE in the column
        end
    end

endmodule
```

2nd) Understanding the modules:

➢ Row and Column summations:

```
module row_col_sum #(
    parameter N,
    parameter DATA_WIDTH = 16
)(
    input clk,
    input rst,
    input [DATA_WIDTH-1:0] A [0:N-1][0:N-1],
    input [DATA_WIDTH-1:0] B [0:N-1][0:N-1],
    output reg [DATA_WIDTH-1:0] row_sum [0:N-1],
    output reg [DATA_WIDTH-1:0] col_sum [0:N-1]
);

    integer i, j;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            for (i = 0; i < N; i = i + 1) begin
                row_sum[i] <= 0;
                col_sum[i] <= 0;
            end
        end else begin
            for (i = 0; i < N; i = i + 1) begin
                row_sum[i] = 0;                    //new row --> reset
                col_sum[i] = 0;
                for (j = 0; j < N/2; j = j + 1) begin
                    row_sum[i] = row_sum[i] + A[i][2*j] * A[i][2*j+1];
                    col_sum[i] = col_sum[i] + B[2*j][i] * B[2*j+1][i];
                end
            end
        end
    end
endmodule
```

- Computes row sums for matrix A and column sums for matrix B, which are required by the Winograd algorithm.
- These sums are used to optimize matrix multiplication by reducing the number of multiplications.

Intermediate Computation Module:

```
module intermediate_computation #(
    parameter N,
    parameter DATA_WIDTH = 16
)(
    input clk,
    input rst,
    input [DATA_WIDTH-1:0] A [0:N-1][0:N-1],
    input [DATA_WIDTH-1:0] B [0:N-1][0:N-1],
    input [DATA_WIDTH-1:0] row_sum [0:N-1],
    input [DATA_WIDTH-1:0] col_sum [0:N-1],
    output reg [2*DATA_WIDTH-1:0] intermediate [0:N-1][0:N-1]
);
    integer i, j, k;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            for (i = 0; i < N; i = i + 1)
                for (j = 0; j < N; j = j + 1)
                    intermediate[i][j] <= 0;
        end else begin
            for (i = 0; i < N; i = i + 1) begin
                for (j = 0; j < N; j = j + 1) begin
                    intermediate[i][j] = -row_sum[i] - col_sum[j];
                    for (k = 0; k < N/2; k = k + 1)
                        intermediate[i][j] = intermediate[i][j] +
                                             (A[i][2*k] + B[2*k+1][j]) *
(A[i][2*k+1] + B[2*k][j]);
                end
            end
        end
    end
endmodule
```

- Uses precomputed row and column sums to generate intermediate results.
- The Winograd multiplication formula involves computing a sum of products using fewer multiplications.
- $C[i][j] = -row_{sum}[i] - col_{sum}[j] + \sum_{k=0}^{\frac{N}{2}-1} (A[i,2k] + B[2k+1,j]) * (A[i,2k+1] + B[2k,j])$

➢ Final Computation Module:

```
module final_computation #(
    parameter N,
    parameter DATA_WIDTH = 16
)(
    input clk,
    input rst,
    input [DATA_WIDTH-1:0] A [0:N-1][0:N-1],
    input [DATA_WIDTH-1:0] B [0:N-1][0:N-1],
    input [2*DATA_WIDTH-1:0] intermediate [0:N-1][0:N-1],
    output reg [2*DATA_WIDTH+3:0] C [0:N-1][0:N-1]
);
    integer i, j;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            for (i = 0; i < N; i = i + 1)
                for (j = 0; j < N; j = j + 1)
                    C[i][j] <= 0;
        end else begin
            // Handle odd matrix case
            if (N % 2 == 1) begin
                for (i = 0; i < N; i = i + 1) begin
                    for (j = 0; j < N; j = j + 1) begin
                        C[i][j] = intermediate[i][j] + A[i][N-1] * B[N-1][j];
                    end
                end
            end else begin
                for (i = 0; i < N; i = i + 1)
                    for (j = 0; j < N; j = j + 1)
                        C[i][j] = intermediate[i][j];
            end
        end
    end
endmodule
```

- Finalizes the computation by adjusting the intermediate results for odd-sized matrices (if needed).
- Produces the final matrix C.
- $C[i][j]=intermediate[i][j]+A[i,N-1]\times B[N-1,j]$

➢ Top-Level Winograd Systolic Array Module:

```
module winograd_systolic_array #(
    parameter N ,
    parameter DATA_WIDTH = 16
)(
    input clk,
    input rst,
    input [DATA_WIDTH-1:0] A [0:N-1][0:N-1],
    input [DATA_WIDTH-1:0] B [0:N-1][0:N-1],
    output [2*DATA_WIDTH+3:0] C [0:N-1][0:N-1]
);
    wire [DATA_WIDTH-1:0] row_sum [0:N-1];
    wire [DATA_WIDTH-1:0] col_sum [0:N-1];
    wire [2*DATA_WIDTH-1:0] intermediate [0:N-1][0:N-1];
    // Instantiate row and column sum module
    row_col_sum #(.N(N), .DATA_WIDTH(DATA_WIDTH)) row_col_sum_inst (
        .clk(clk),
        .rst(rst),
        .A(A),
        .B(B),
        .row_sum(row_sum),
        .col_sum(col_sum)
    );
    // Instantiate intermediate computation module
    intermediate_computation #(.N(N), .DATA_WIDTH(DATA_WIDTH))
intermediate_comp_inst (
        .clk(clk),
        .rst(rst),
        .A(A),
        .B(B),
        .row_sum(row_sum),
        .col_sum(col_sum),
        .intermediate(intermediate)
    );
    // Instantiate final computation module
    final_computation #(.N(N), .DATA_WIDTH(DATA_WIDTH)) final_comp_inst (
        .clk(clk),
        .rst(rst),
        .A(A),
        .B(B),
        .intermediate(intermediate),
        .C(C)
    );
endmodule
```

- The main module that connects all submodules.
- Controls the computation flow and synchronizes all processing stages.

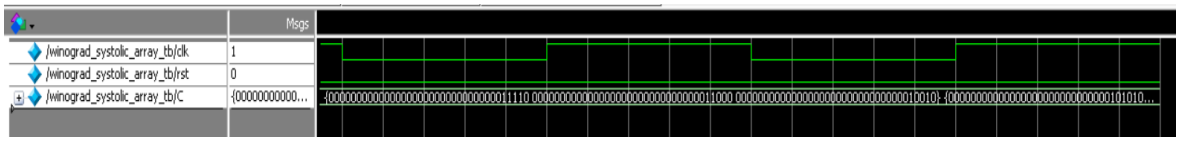
➔ Finally, we must test our module by writing a test bench:

```
`timescale 1ns / 1ps

module winograd_systolic_array_tb;

    parameter N = 3;            // to test for both 2x2 matrix and 3x3 matrix
    parameter DATA_WIDTH = 16;  // Bit width for each matrix element

    reg clk, rst;
    reg [DATA_WIDTH-1:0] A [0:N-1][0:N-1];
    reg [DATA_WIDTH-1:0] B [0:N-1][0:N-1];
    wire [2*DATA_WIDTH+3:0] C [0:N-1][0:N-1];

    winograd_systolic_array #(.N(N), .DATA_WIDTH(DATA_WIDTH)) dut (
        .clk(clk),
        .rst(rst),
        .A(A),
        .B(B),
        .C(C)
    );

    always #10 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;

        #20 rst = 0;

        // Initialize Matrices A and B based on N
        if (N == 2) begin
            A[0][0] = 8'd1;  A[0][1] = 8'd2;
            A[1][0] = 8'd3;  A[1][1] = 8'd4;

            B[0][0] = 8'd5;  B[0][1] = 8'd6;
            B[1][0] = 8'd7;  B[1][1] = 8'd8;
        end
```

```
        else if (N == 3) begin
            A[0][0] = 8'd1;  A[0][1] = 8'd2;  A[0][2] = 8'd3;
            A[1][0] = 8'd4;  A[1][1] = 8'd5;  A[1][2] = 8'd6;
            A[2][0] = 8'd7;  A[2][1] = 8'd8;  A[2][2] = 8'd9;

            B[0][0] = 8'd9;  B[0][1] = 8'd8;  B[0][2] = 8'd7;
            B[1][0] = 8'd6;  B[1][1] = 8'd5;  B[1][2] = 8'd4;
            B[2][0] = 8'd3;  B[2][1] = 8'd2;  B[2][2] = 8'd1;
        end

        // Print input matrices
        $display("Input Matrix A:");
        for (int i = 0; i < N; i = i + 1) begin
            for (int j = 0; j < N; j = j + 1)
                $write("%d ", A[i][j]);
            $write("\n");
        end

        $display("Input Matrix B:");
        for (int i = 0; i < N; i = i + 1) begin
            for (int j = 0; j < N; j = j + 1)
                $write("%d ", B[i][j]);
            $write("\n");
        end

        // Wait for systolic array processing to complete
        #100;

        // Print Output Matrix C
        $display("Output Matrix C:");
        for (int i = 0; i < N; i = i + 1) begin
            for (int j = 0; j < N; j = j + 1)
                $write("%d ", C[i][j]);
            $write("\n");
        end

        $stop;
    end

endmodule
```

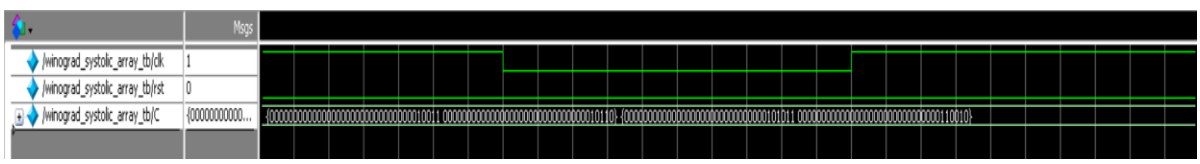
➢ **Timing Analysis:**

1) For N = 3 (3x3 matrix):




```
# Input Matrix A:
#      1      2      3
#      4      5      6
#      7      8      9
# Input Matrix B:
#      9      8      7
#      6      5      4
#      3      2      1
# Output Matrix C:
#          30          24          18
#          84          69          54
#         138         114          90
```

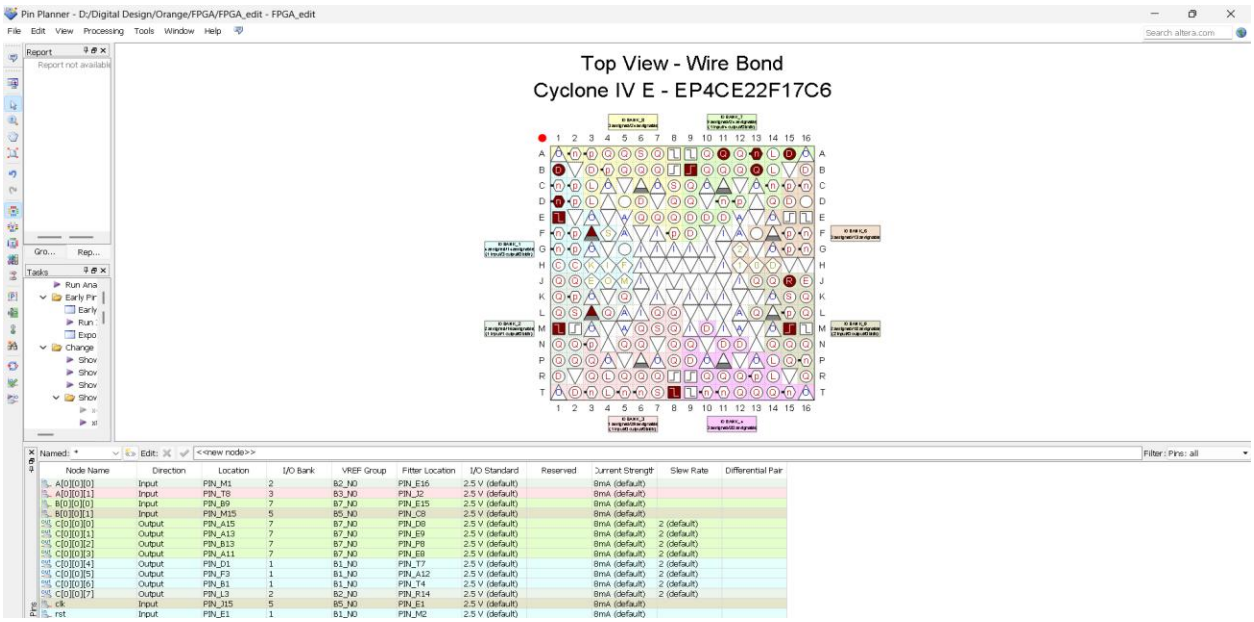
2) For N = 2 (2x2 matrix):




```
# Input Matrix A:
#      1      2
#      3      4
# Input Matrix B:
#      5      6
#      7      8
# Output Matrix C:
#          19          22
#          43          50
```

➢ Implementation on FPGA:

Due to having many inputs and outputs N is assigned to equal 1 and data width = 2 to be able to assign all the ports to pins on the FPGA.



➢ Analysis and Synthesis Summary:

Analysis & Synthesis Summary	
Analysis & Synthesis Status	Successful - Thu Feb 20 18:22:29 2025
Quartus II 32-bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	FPGA_edit
Top-level Entity Name	winograd_systolic_array
Family	Cyclone IV E
Total logic elements	4
Total combinational functions	4
Dedicated logic registers	4
Total registers	4
Total pins	14
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total PLLs	0

Unconstrained Paths

	Property	Setup	Hold
1	Illegal Clocks	0	0
2	Unconstrained Clocks	0	0
3	Unconstrained Input Ports	5	5
4	Unconstrained Input Port Paths	18	18
5	Unconstrained Output Ports	4	4
6	Unconstrained Output Port Paths	4	4

➢ Setup and hold time:

Summary (Setup)

	Clock	Slack	End Point TNS
1	clk	13.419	0.000

Summary (Hold)

	Clock	Slack	End Point TNS
1	clk	-0.493	-1.875

➢ After applying constraints:

Clocks Summary

	Clock Name	Type	Period	Frequency	Rise	Fall	Duty Cycle	Divide by	Multiply by	Phase	Offset	Edge List	Edge Shift	Inverted	Master	Source	Targets
1	clk	Base	20.000	50.0 MHz	0.000	10.000											{ clk }

Unconstrained Paths Summary

	Property	Setup	Hold
1	Illegal Clocks	0	0
2	Unconstrained Clocks	0	0
3	Unconstrained Input Ports	0	0
4	Unconstrained Input Port Paths	0	0
5	Unconstrained Output Ports	0	0
6	Unconstrained Output Port Paths	0	0

Summary

	Check	Number of Issues Found
1	no_clock	0
2	multiple_clock	0
3	pos_neg_clock_domain	0
4	generated_clock	0
5	virtual_clock	1
6	no_input_delay	0
7	no_output_delay	0
8	partial_input_delay	0
9	partial_output_delay	0
10	io_min_max_delay_consistency	26
11	reference_pin	0
12	generated_io_delay	0
13	latency_override	0
14	partial_multicycle	0
15	multicycle_consistency	0
16	loops	0
17	latches	0
18	pll_cross_check	0
19	uncertainty	1
20	partial_min_max_delay	0
21	clock_assignments_on_output_ports	0
22	input_delay_assigned_to_clock	0

➢ Setup Timing Report:

Report Timing

Command Info | Summary of Paths

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	13.419	final_computation:final_comp_inst|C[0][0][0]	C[0][0][0]	clk	clk	20.000	-2.394	3.987
2	13.599	final_computation:final_comp_inst|C[0][0][3]	C[0][0][3]	clk	clk	20.000	-2.394	3.807
3	14.016	final_computation:final_comp_inst|C[0][0][1]	C[0][0][1]	clk	clk	20.000	-2.394	3.390
4	14.021	final_computation:final_comp_inst|C[0][0][2]	C[0][0][2]	clk	clk	20.000	-2.394	3.385
5	19.333	A[0][0][1]	final_computation:final_comp_inst|C[0][0][1]	clk	clk	20.000	2.295	2.777
6	19.360	A[0][0][1]	final_computation:final_comp_inst|C[0][0][2]	clk	clk	20.000	2.295	2.750

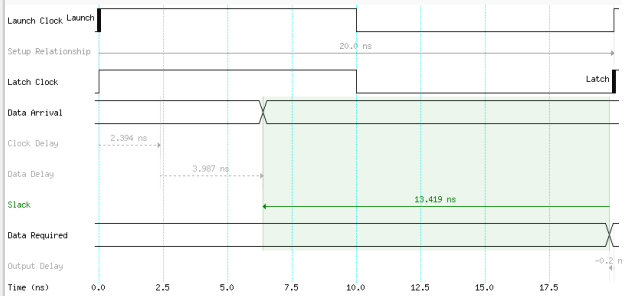
Path #1: Setup slack is 13.419

Path Summary | Statistics | Data Path | Waveform | Extra Fitter Information

	Property	Value
1	From Node	final_computation:final_comp_inst|C[0][0][0]
2	To Node	C[0][0][0]
3	Launch Clock	clk
4	Latch Clock	clk
5	Data Arrival Time	6.381
6	Data Required Time	19.800
7	Slack	13.419

Path #1: Setup slack is 13.419

Path Summary | Statistics | Data Path | Waveform | Extra Fitter Information



➢ Hold Timing Report

Report Timing

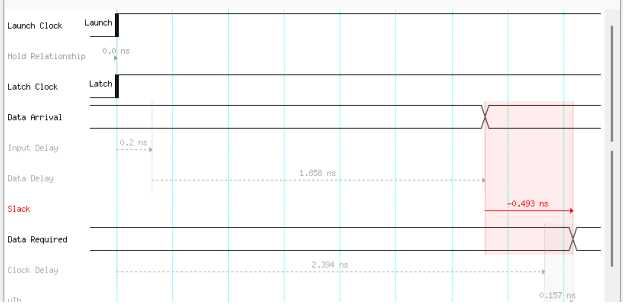
Command Info | Summary of Paths

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-0.493	B[0][0][1]	final_computation:final_comp_inst|C[0][0][3]	clk	clk	0.000	2.394	1.858
2	-0.490	B[0][0][1]	final_computation:final_comp_inst|C[0][0][2]	clk	clk	0.000	2.394	1.861
3	-0.489	B[0][0][1]	final_computation:final_comp_inst|C[0][0][1]	clk	clk	0.000	2.394	1.862
4	-0.403	B[0][0][0]	final_computation:final_comp_inst|C[0][0][0]	clk	clk	0.000	2.394	1.948
5	-0.326	B[0][0][0]	final_computation:final_comp_inst|C[0][0][1]	clk	clk	0.000	2.394	2.025
6	-0.318	B[0][0][0]	final_computation:final_comp_inst|C[0][0][3]	clk	clk	0.000	2.394	2.033

Path #1: Hold slack is -0.493 (VIOLATED)

Path Summary | Statistics | Data Path | Waveform | Extra Fitter Information

	Property	Value
1	From Node	B[0][0][1]
2	To Node	final_computation:final_comp_inst|C[0][0][3]
3	Launch Clock	clk
4	Latch Clock	clk
5	Data Arrival Time	2.058
6	Data Required Time	2.551
7	Slack	-0.493 (VIOLATED)

Path #1: Hold slack is -0.493 (VIOLATED)

Path Summary | Statistics | Data Path | Waveform | Extra Fitter Information